

Is Your AWR/ Statspack Report Relevant?

In December, at UKOUG Tech15, I presented how I read an AWR report (or a Statspack report if you don't have Diagnostic Pack) and go straight to the goal: root cause, recommendations, and estimation of gain.

Franck Pachot, dbi services ♠

Some of them have no valuable information. Before starting to read an AWR/Statspack report, there are a few points that I quickly check before anything else, because I don't want to waste time on an irrelevant report. The risk if you don't check report relevancy is misinterpretation and drawing wrong conclusions.

Report Duration & Captured SQL

Oracle is heavily instrumented and measures statistics in real time about the database activity. The numbers recorded are cumulated values since the instance startup. In order to see what happens during a specific time window, those statistics are stored in snapshots and the AWR/Statspack report is an easy way to view the delta values between two snapshots.

An AWR report shows the activity during a time window. Most of the statistics

do not have more details through the time dimension (an exception is the wait event histograms). We have only the average value during the elapsed time. As a consequence, if the time window is too wide, then the averages will hide interesting details. Let's take an example. If your system is 100% busy during one hour and then idle for the 4 remaining hours, then a report covering the whole 5 hours will show a 80% idle system. And you will never be able to address the busy activity properly – which is your goal.

There is another reason to avoid reports that cover a long period. Important information is detailed for the statements that were still in the shared pool at the time of the end snapshot. If the duration is too large and you have a lot of heterogeneous activity, then you will probably miss a lot of SQL statements.

Let's check that from the first section of my example AWR report:

	Snap Id	Snap Time	Sessions	Curs/Sess
Begin Snap:	330	11-Mar-14 15:18:53	40	5.4
End Snap:	336	11-Mar-14 15:34:32	55	4.7
Elapsed:		15.64 (mins)		
DB Time:		76.48 (mins)		

Here my report duration is 15 minutes. It's the most important information. All the performance analysis will be about what the database was doing during that time.

To be more precise, we will analyse:

- What the users were doing on the database: Users are probably running some SQL or PL/SQL, and then parse it, execute it, or fetch from it. Or running Java, etc.
- Which resources the database has consumed to serve those user calls. Of course the Oracle software run in CPU, but can also wait on some system calls (I/O for example).

So this is the first thing to check when you have an AWR/Statspack report: the elapsed time must cover your performance issue. And it should cover an homogeneous activity.

Which duration is good? Let's take some examples. Several years ago, I was given some daily reports covering 24 hours and was asked to analyse the database performance from that. My answer to that was a simple one: I can't. During 24 hours, the database has lot of idle activity. This will lower all the averages. As an analogy, imagine you look at the highway traffic and base your analysis only on the number of cars that went through each day. The average traffic over 24 hours will be very low. Obviously, you need to count the cars at peak hours if you want to base your analysis on relevant numbers.

If your performance issue occurs during a 10 minute peak, then you have to get manual snapshots covering 10 minutes. The default hourly snapshots are not sufficient here. Taking manual snapshots is as easy as calling `dbms_workload_repository.create_snapshot`. If the performance issue is very short (1 minutes for example) then probably you want another tool, such as ASH, because taking AWR snapshot has an overhead that will be included in your report.

Another example: you have a long batch job that takes 10 hours to complete. If you run a report covering those 10 hours, you will cover the activity, but the activity is not uniform. You have times where you are doing lot of I/O, other times where you are 100% in CPU, and other time where you're just waiting for something else. Once again, the averages will hide everything. And anyway, when you will need to go to the SQL details, then you will not get them.

Look at the following Time Model section:

```
SQL ordered by Elapsed Time          DB/Inst: ORCL/orcl  Snaps: 330-336
...
-> Captured SQL account for 94.6% of Total DB Time (s): 4,589
-> Captured PL/SQL account for 94.5% of Total DB Time (s): 4,589
```

Here I know I'll have details on 94% of the SQL activity, which is good. If you see a low percentage here, you'll miss the most interesting details. And that can be a consequence of a report covering a period that is too large, or not using bind variables. The SQL statements that had run at the beginning of the job have been aged out of the shared pool before the end, then they are not present in the report.

So how to analyse the performance of a 10 hours job? Just get the 10 reports covering each hour. You will have to analyse all of them. Of course if you know exactly which phase of the job is longer than expected, then you can focus on the few reports related to that.

Anyway, you should always interview the users. You will see a lot of background activity in the report, which you don't want to tune. Sit with the end-users in front of the database activity graph. You have it from Enterprise Manager or any other tool (I've scripts to export some AWR or Statspack data to a nice Excel graph). See where you have activity and identify the time windows where you have homogenous activity that you want to tune.

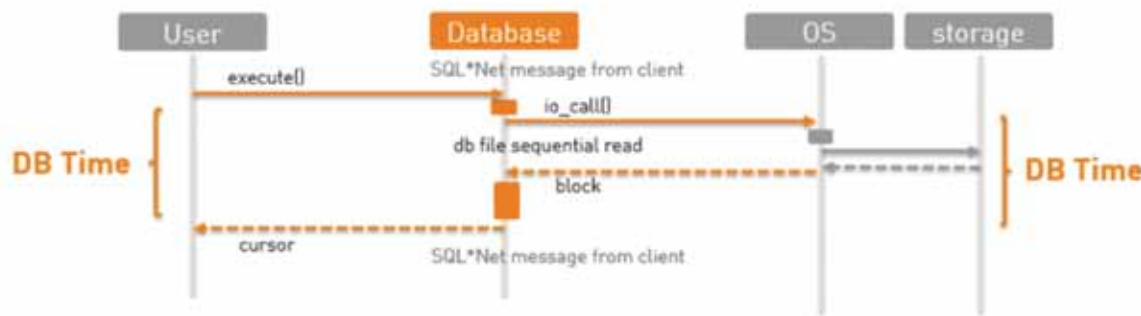
Database Load (DB time & AAS)

Once you verified that the elapsed time covers a relevant duration, you want to know if your database was busy during that time.

Most of the time, a user is doing nothing with the database: he is reading the last result that was fetched. Or the application server is computing some stuff before calling back to the database. Or the user is still connected but is at the coffee machine. All that is idle time for the database. So the DB time – which is the time spent in the database – can be lower than the elapsed time.

On the other hand, the database can be accessed concurrently by several users (sessions) and can process many calls at the same time. For one session, the DB time is between 0 (totally idle) and elapsed time (totally busy). But for the whole system, the DB time can be larger than the elapsed time because it is multi-user software.

The DB time is the most important for us because it's the only way to match the end-user performance feeling (response time) with the system usage. From the user point of view, the DB time is the sum of all user calls to the database, from the end of 'SQL*Net message from client' to the next 'SQL*Net message to client'. From the system point of view, the DB time is the sum of system usage, which is the CPU usage plus any system call to lower layers (storage, network, etc). Application waits (latches, locks, `dbms_lock.sleep`,... are implemented as wait events as well, with the exception of spinning latches that loop in CPU.



Above is a sequence diagram to show DB time from user and system point of view.

Our tuning job is all about analysing the database activity. The main unit is the time, which is what we want to reduce. And the AWR report is the balance sheet among resources and their use. We read it to understand where we can improve the user response time.

There is no point to analyse database performance if the database is just idle. Don't waste time on AWR/Statspack report from an idle database. The first thing to do is to check the database load by dividing DB time by elapsed time:

$$\text{DB time} / \text{Elapsed} = 76.48 / 15.64 = 4.9$$

On average I have 4.9 active sessions. It is 4.9 users running something on the database if I'm in client/server, or it is 4.9 active connections from a connection pool. Always keep in mind that it's an average: I may also have 49 end-users that are spending 10% of their time only on database calls. Or one busy all the time, and 39 ones busy at 10%...

That's my database load, telling me whether my database is busy or not. Busy means running or waiting on something that must be completed before being able to run. If DB time is much lower than the sum of the user experienced response time, then the performance issue is probably not at database level. That is our first analysis: when somebody comes to us saying that the database is slow, we need first to check if that 'slow' activity is actually in the database or not.

Always check the database load calculated from DB time divided by elapsed time. And compare it with the user activity that is expected to run on database.

This ratio is also called 'Average Active Session' or 'AAS'. I call it database load because it's exactly the same idea as the UNIX load average which counts the average number of processes running or willing to run on CPU. But here it concerns all

database resources: not only CPU but also I/O, network, or even application waits (locks). Note that in Linux, I/O waits are included in load average, but not all waits.

Foreground Events & CPU

From the user point of view we have the time model section that qualify the user call. But when analysing the database resources that are used to serve that call, the main entry is the following section (see Figure A below).

A 'Foreground Event' which was called 'Timed Event' in 11g, is the part of DB time where the session process is either:

- Running in CPU (which does not include waiting in the host runqueue)
- Waiting for a system call – known from Oracle as wait event (which may include the time in runqueue at the end of the call).

It's important to know that DB CPU do not include the time in runqueue, which is when a process has something to run on CPU but no CPU is available. This explains why the sum of those timed events can be lower than the total DB time. And this is important to know because only some part of the runqueue is not accounted here. When the system is suffering CPU starvation, then all the wait events are inflated because they include the time in runqueue at the end of the call. The reason is simply that the statistics are updated by CPU instructions – so the counter can't be stopped until the process is back running in CPU.

What you need to remember is that when all cores are busy, then the wait events are inflated, thus becoming meaningless. And having the sum of timed events lower than the DB time is usually a sign of this situation. In this case, you have to analyse the system performance at server (and/or hypervisor if virtualised) level before wasting your time on an AWR report. Except some bugs where some system calls are not instrumented properly, the time that is not accounted here is the time waiting in the host runqueue in the middle of CPU

Top 10 Foreground Events by Total Wait Time

Event	Waits	Total Wait Time (sec)	Wait Avg (ms)	% DB Wait time Class
db file sequential read	31,762	3,497	110	76.2 User I/O
DB CPU		608		13.3
log file sync	5,676	189	33	4.1 Commit
enq: TX - row lock contention	139	171	1229	3.7 Applicatio
control file sequential read	1,105	9	8	.2 System I/O

FIGURE A

Host CPU			Load Average		%User	%System	%WIO	%Idle
CPUs	Cores	Sockets	Begin	End				
8	4	1	1.5	1.1	11.2	3.5	49.2	85.2

FIGURE B

operations. Then if the total is far less than 100% it's probably a sign of CPU starvation. And because CPU starvation inflates the wait events, making their duration irrelevant, we need to check the host CPU utilisation before continuing with wait events. Fortunately, we have that information in the report as well.

Host CPU

When processes are going to runqueue after a system call, the time waiting for CPU is accounted in the wait event. And that means that the wait event analysis is totally biased. In that case we must address the host CPU issue first. You can use 'sar' on the server to see the CPU load detail. Here I'll take the numbers from the 'Host CPU' section of the AWR or Statspack report.

But be careful, in both cases, the percentages of CPU utilisation are misleading. One thing to consider here is that the CPU utilisation reported is related to the 8 CPUs, but we have

physically 4 cores hyper-threaded. Hyper-threading will never double the CPU time available, so it's safer to convert the percentages to be related to the number of cores.

I never consider CPU utilisation percentages as reported by the OS without converting them to be related to the number of cores. It's misleading. I've seen people thinking their system was mainly idle because they had 25% utilisation. But that was on a 4-way SMT. All the cores were 100% busy.

In my example here, the load average is much lower than the number of cores so I'm quite sure that the processes do not wait in the runqueue when they return from the system call. Check it with 'sar -q' on the host. If you see that the load average reaches the number of cores, then you should analyse the host usage before going to the database AWR/Statspack report. Note that the host CPU usage can come from outside of your database: other instances, OEM agents, etc.

Conclusion

Here I have read a few sections from an AWR report, but I've not analysed anything yet. I will have to look at the major timed events, and get more details from the relevant sections, in order to find the root cause. This is what I explained in December in my 'Reading an AWR report: straight to the goal' Tech15 session. The Prezi is online: goo.gl/HcUxvL

Here I've done the major pre-checks to ensure that my report contains relevant information to be analysed, and it takes only 5 minutes. To summarise, I checked that:

- › The duration covers a period of homogenous activity representative of my performance issue
- › The captured SQL statements cover the major part of SQL and PL/SQL execution time
- › The database was active at that time and database load matches what I expect from user activity
- › The host was not in CPU starvation, so the wait events numbers are measuring only the system calls, and timed events detail the whole DB time.

I do this every time I have to analyse an AWR or Statspack report. In my opinion, not doing those pre-checks exposes you to a lot of possible misinterpretation. It's quick and makes your analysis safer. ■



ABOUT THE AUTHOR

Franck Pachot

Senior Consultant, dbi-services

Franck is senior consultant and Oracle technology leader at dbi services in Switzerland. He has 20 years of experience in Oracle databases, all areas from development, data modeling, performance, administration, and training. Oracle Certified Master and Oracle ACE, he tries to leverage knowledge sharing in forums, publications and presentations.

Blog: blog.pachot.net



ch.linkedin.com/in/franckpachot



@FranckPachot